



Instruction Guide

Implementation Guide for Ascon-128 on CW305

Vivado / Vitis HLS / ChipWhisperer workflow

Prepared by

Rupesh Hemant Bhusare

2203106, CSE

Contents

1	Purpose of This Guide	4
2	Required Hardware and Software	5
2.1	Hardware	5
2.2	Software	5
3	Folder and File Preparation	7
3.1	Recommended Working Structure	7
3.2	Files to Copy from Ascon Reference Source	7
3.3	Local Files Already Available	8
4	Getting the Ascon Reference Code	9
4.1	Why Use <code>crypto_aead/ascon128v12/ref</code>	9
5	Making the Code HLS-Friendly	10
5.1	Merge Encrypt and Decrypt	10
5.2	Why Merge Them	10
5.3	Add HLS Pragmas	11
5.4	Files to Keep	11
6	Creating the Vitis HLS Project	12
6.1	GUI Method	12
6.2	Tcl Method	12
7	Writing and Running <code>run_exploration.tcl</code>	13
7.1	Create the Script	13

7.2	Run the Script	14
7.3	What Each Solution Does	14
8	Reading HLS Reports	15
8.1	Important Rule for This BTP	15
9	Exporting RTL / IP	16
9.1	From GUI	16
9.2	What to Save	16
10	Building the FPGA Bitstream for CW305	17
10.1	GUI Steps	17
11	Future Work: Running Hardware Test on CW305	19
11.1	Notebook Path	19
11.2	Planned Command Flow from the Notebook	19
11.3	Safe Disconnect	20
12	Future Work: Setting Up Trace Capture for CPA	21
12.1	Basic Capture Rules	21
13	Future Work: Running CPA with Hamming Distance	22
13.1	Attack Logic	22
13.2	Pseudo-code	22
13.3	Expected Output	23
14	Future Work: Comparing HLS and RTL Results	24
14.1	Fair Method	24
15	Common Errors and Fixes	25
16	Recommended Experiment Order	26
17	Appendix: Commands, Paths, and Checklists	27
17.1	Useful Paths	27
17.2	Useful Commands	27

17.3 Before Synthesis Checklist	27
17.4 Before Future Trace Capture Checklist	28

Chapter 1

Purpose of This Guide

This guide gives a direct step-by-step path to build the Ascon-128 hardware target for the ChipWhisperer CW305 board. It is based on the local files present in the current project folder and is written so that the completed HLS and bitstream-generation stages can be repeated in a controlled way.

This guide covers:

- selecting the correct Ascon source folder,
- making the code HLS-friendly,
- creating the Vitis HLS project,
- running the design-space exploration Tcl script,
- reading HLS reports,
- exporting RTL or IP,
- building the FPGA bitstream for CW305.

Everything after bitstream generation is included only as future methodology, not as completed practical work.

Chapter 2

Required Hardware and Software

2.1 Hardware

Table 2.1: Required hardware

Item	Purpose
ChipWhisperer CW305 Artix FPGA Target	Main FPGA target for Ascon hardware
ChipWhisperer capture setup	Future power trace capture and trigger handling
USB cable(s)	CW305 connection and programming
Host computer	Runs Vivado, Vitis HLS, Python, and notebook tools

2.2 Software

Table 2.2: Required software

Tool	Purpose
Vivado 2024.2	FPGA build flow
Vitis HLS 2024.2	C to RTL conversion and design exploration
Python with ChipWhisperer package	Future board control and trace capture
Jupyter Notebook	Future hardware test and attack scripts

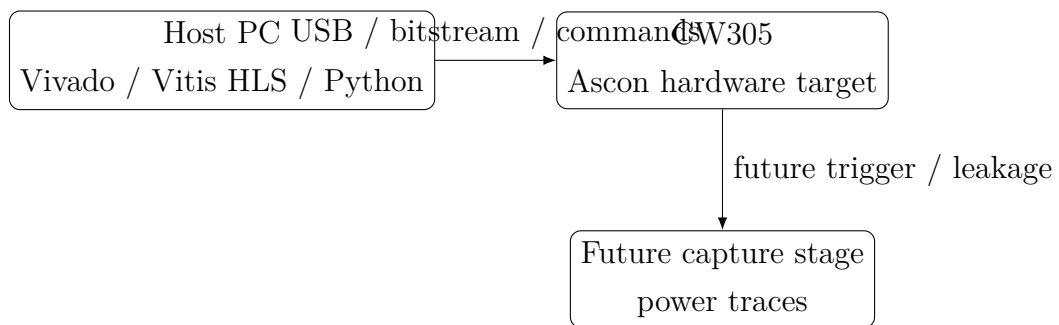


Figure 2.1: Basic lab setup and future measurement path

Chapter 3

Folder and File Preparation

The current project folder contains planning and source context files, but it does not contain the full original Ascon reference directory or a complete Vivado project. Prepare your work area with a clean hardware source folder.

3.1 Recommended Working Structure

```
Desktop/Ascon_DSE_Project/  
  ascon_src/  
  hls_project/  
  vivado_project/  
  notebooks/
```

3.2 Files to Copy from Ascon Reference Source

Go to the extracted Ascon repository and select:

```
ascon-c-main/crypto_aead/ascon128v12/ref
```

Copy at least these files into your hardware source folder:

Table 3.1: Reference files to start from

File	Purpose
<code>encrypt.c</code>	Original encryption logic
<code>decrypt.c</code>	Original decryption logic
<code>api.h</code>	API and size constants
<code>ascon.h</code>	Main algorithm header if required by the selected version
<code>permutations.h</code>	Permutation support and state handling
<code>word.h</code> / <code>config.h</code>	Helper files if present in the selected source tree

3.3 Local Files Already Available

The current project folder already gives useful prepared material:

- `ascon_modified_codes.txt`: merged code draft,
- `run_exploration_tcl.txt`: HLS Tcl automation,
- `Ascon_Hardware_Test.ipynb`: prepared future hardware test notebook.

Chapter 4

Getting the Ascon Reference Code

4.1 Why Use `crypto_aead/ascon128v12/ref`

Use:

```
crypto_aead/ascon128v12/ref
```

Reason:

1. It is the standard reference implementation.
2. It is clean C code.
3. HLS tools handle clean C much better than CPU-specific optimized code.
4. It is easier to explain in an academic report.

Table 4.1: Why the reference folder is selected

Folder type		Main property	Suitability for HLS
ref		Clean and simple C code	Best choice
Optimized folders	CPU	More low-level tricks	Harder to synthesize and explain

Chapter 5

Making the Code HLS-Friendly

5.1 Merge Encrypt and Decrypt

The local prepared source already follows the correct idea: one top function with a mode input.

```
int ascon_core(unsigned char* d_out, unsigned long long*
    d_out_len,
                const unsigned char* d_in, unsigned long long
                d_in_len,
                const unsigned char* ad, unsigned long long adlen,
                const unsigned char* nsec, const unsigned char*
                npub,
                const unsigned char* k, int mode);
```

Use:

- mode = 0 for encryption,
- mode = 1 for decryption.

5.2 Why Merge Them

1. One hardware module is easier to integrate.
2. Shared permutation logic can be reused.
3. It matches a practical hardware IP style.
4. It is easier to compare one consistent core across HLS solutions.

5.3 Add HLS Pragas

Use the same kind of pragmas shown in the local merged code:

```
#pragma HLS INTERFACE ap_memory port=d_in depth=100
#pragma HLS INTERFACE ap_memory port=d_out depth=100
#pragma HLS INTERFACE ap_memory port=ad depth=100
#pragma HLS INTERFACE ap_memory port=npub depth=16
#pragma HLS INTERFACE ap_memory port=k depth=16
```

5.4 Files to Keep

Recommended source set for HLS:

- `api.h`
- `permutations.h`
- `ascon_core.c`
- `main.c` for testbench or simulation-style checking

Chapter 6

Creating the Vitis HLS Project

6.1 GUI Method

1. Open Vitis HLS 2024.2.
2. Click **Create Project**.
3. Set project name to `ascon_hardware_dse`.
4. Add `ascon_core.c` and required headers.
5. Set top function to `ascon_core`.
6. Select FPGA part xc7a100tftg256-2.
7. Set clock period to 10 ns.
8. Finish project creation.

6.2 Tcl Method

You can create and run the project with Tcl. The local file already provides the script pattern.

```
open_project ascon_hardware_dse
set_top ascon_core
add_files ascon_core.c
```

Chapter 7

Writing and Running `run_exploration.tcl`

7.1 Create the Script

Create a file named:

```
run_exploration.tcl
```

Use this content based on the local file:

```
open_project ascon_hardware_dse
set_top ascon_core
add_files ascon_core.c

open_solution "solution1_baseline"
set_part {xc7a100tftg256-2}
create_clock -period 10 -name default
csynth_design

open_solution "solution2_unrolled"
set_part {xc7a100tftg256-2}
create_clock -period 10 -name default
set_directive_unroll -factor 2 "ascon_core/data_loop"
csynth_design

open_solution "solution3_pipelined"
set_part {xc7a100tftg256-2}
create_clock -period 10 -name default
set_directive_pipeline "ascon_core/data_loop"
```

```

csynth_design

open_solution "solution4_dataflow"
set_part {xc7a100tftg256-2}
create_clock -period 10 -name default
set_directive_dataflow "ascon_core"
csynth_design

close_project
exit

```

7.2 Run the Script

Terminal command:

```
vitis_hls -f run_exploration.tcl
```

7.3 What Each Solution Does

Table 7.1: Meaning of the HLS solutions

Solution	Directive	Meaning
Baseline	none	Direct reference point
Unrolled	unroll factor 2	More parallel loop body hardware
Pipelined	pipeline	Better overlap between iterations
Dataflow	dataflow	More concurrency across stages

Chapter 8

Reading HLS Reports

After synthesis, open each solution report and note:

1. latency,
2. initiation interval,
3. LUT count,
4. FF count,
5. BRAM count,
6. timing estimate.

8.1 Important Rule for This BTP

For leakage comparison, do not look only at performance. Also record structural changes that may change switching activity and later side-channel leakage.

Chapter 9

Exporting RTL / IP

9.1 From GUI

1. Open the chosen HLS solution.
2. Click **Export RTL**.
3. Choose Verilog output.
4. If needed, package as IP for Vivado integration.

9.2 What to Save

Save:

- generated Verilog,
- synthesis reports,
- interface summary,
- latency report,
- packaged IP files if used.

Chapter 10

Building the FPGA Bitstream for CW305

The current workspace does not contain a full Vivado project, so this section records the required build steps up to bitstream generation.

10.1 GUI Steps

1. Open Vivado 2024.2.
2. Create or open the CW305 wrapper project.
3. Add the generated HLS RTL or packaged IP.

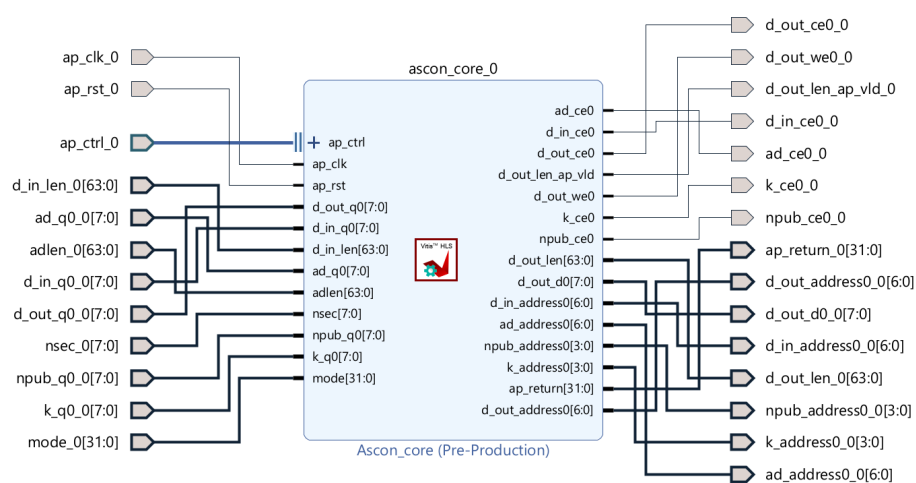


Figure 10.1: IP core interface diagram for the generated `ascon_core` block, used as a reference before MIG and UART integration in Vivado Block Design.

4. Do the MIG + UART integration in the Vivado Block Design so the IP core is connected correctly with the required memory and communication path.
5. Connect the design to the CW305 wrapper signals.
6. Run synthesis.
7. Run implementation.
8. Generate bitstream.
9. Save the final `.bit` file as `cw305_top.bit` or update the notebook path.

Chapter 11

Future Work: Running Hardware Test on CW305

11.1 Notebook Path

The current project folder already contains:

```
Ascon_Hardware_Test.ipynb
```

11.2 Planned Command Flow from the Notebook

```
import chipwhisperer as cw
import time
import binascii

target = cw.target(None, cw.targets.CW305)
target.fpga.FPGAProgram(open("cw305_top.bit", "rb").read())

target.vccint_set(1.0)
target pll.enable_set(True)
target pll.outenable_set(True, 1)
target pll.outfreq_set(10E6, 1)
time.sleep(0.1)

secret_key = bytearray([0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06,
                        0x07,
                        0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E,
                        0x0F])
```

```
plaintext = bytearray([0x48, 0x45, 0x4C, 0x4C, 0x4F, 0x20, 0x41,
                        0x53,
                        0x43, 0x4F, 0x4E, 0x20, 0x31, 0x32, 0x33,
                        0x21])

target.fpga_write(target.REG_CRYPT_KEY, secret_key)
target.fpga_write(target.REG_CRYPT_TEXTIN, plaintext)
target.go()
ciphertext = target.fpga_read(target.REG_CRYPT_CIPHEROUT, 16)
```

11.3 Safe Disconnect

At the end, disconnect cleanly:

```
target.dis()
```

Note: the saved notebook output shows a `NameError` in one disconnect cell because `target` was not defined in that stored execution. In the future practical run, execute all setup cells first.

Chapter 12

Future Work: Setting Up Trace Capture for CPA

12.1 Basic Capture Rules

1. Keep the key fixed for one attack run.
2. Vary the plaintext or chosen input data.
3. Keep the clock constant across experiments.
4. Keep gain and sample length constant when comparing HLS and RTL.
5. Save trace metadata with each run.

At the time of writing this guide, the physical CW305 trace-capture stage is still future work. When this stage begins, the implementation name, clock, trace count, sample count, trigger source, and input pattern must all be recorded in a separate lab log so that later CPA results are reproducible and fair.

Chapter 13

Future Work: Running CPA with Hamming Distance

13.1 Attack Logic

1. Choose the internal transition to model.
2. For each input, compute predicted old and new state values.
3. Compute Hamming Distance.
4. Correlate predicted leakage with measured traces.
5. Find the strongest peak and rank the guesses.

13.2 Pseudo-code

```
def hd(a, b):  
    return bin(a ^ b).count("1")  
  
for guess in guesses:  
    hypothesis = []  
    for sample_input in inputs:  
        old_state, new_state = model_transition(sample_input,  
                                                guess)  
        hypothesis.append(hd(old_state, new_state))  
  
    for t in range(num_points):  
        corr[guess][t] = pearson(hypothesis, traces[:, t])
```

13.3 Expected Output

When this future stage is executed, the outputs should include the correlation plot, best guess ranking, trace count used, and the exact target transition used in the model.

Chapter 14

Future Work: Comparing HLS and RTL Results

14.1 Fair Method

1. Same algorithm,
2. same board family,
3. same clock,
4. same input plan,
5. same trace count rule,
6. same attack script.

At the time of writing this guide, the RTL implementation and its power-analysis results are future work. The comparison should be performed only after both HLS and RTL implementations are tested under the same measurement conditions.

Chapter 15

Common Errors and Fixes

Table 15.1: Common issues and fixes

Problem	Likely reason	Fix
HLS top function not found	Wrong top name	Set top function to <code>ascon_core</code>
Pointer/interface errors	Missing pragmas or unsupported style	Use simple interfaces and explicit pragmas
Wrong FPGA part	Board mismatch	Set part to <code>xc7a100tftg256-2</code> if that is your board target
Bitstream not loading	Wrong path or failed build	Check <code>cw305_top.bit</code> path and build status
Notebook disconnect error	<code>target</code> not created in current run	Execute setup cells first
Weak CPA result later	Wrong leakage model or poor alignment	Check trigger, model, and trace quality

Chapter 16

Recommended Experiment Order

1. Prepare source files from `ascon128v12/ref`.
2. Build merged `ascon_core`.
3. Run HLS baseline solution.
4. Run HLS exploration solutions.
5. Validate hardware function in simulation.
6. Export RTL and build the CW305 bitstream.
7. In future work, run notebook-based hardware test.
8. In future work, capture traces.
9. In future work, run CPA with the HD model.
10. In future work, repeat the study for RTL implementation.
11. In future work, compare HLS and RTL results under matched conditions.

Chapter 17

Appendix: Commands, Paths, and Checklists

17.1 Useful Paths

```
Current working folder:  
Desktop/Ascon_DSE_Project  
  
Current local context files:  
Desktop/Ascon_DSE_Project/ascon_modified_codes.txt  
Desktop/Ascon_DSE_Project/run_exploration_tcl.txt  
Desktop/Ascon_DSE_Project/Ascon_Hardware_Test.ipynb
```

17.2 Useful Commands

```
vitis_hls -f run_exploration.tcl
```

17.3 Before Synthesis Checklist

- top function is `ascon_core`,
- all required headers are added,
- FPGA part is correct,
- clock period is set,
- testbench compiles.

17.4 Before Future Trace Capture Checklist

- board is detected,
- bitstream is loaded,
- clock is stable,
- trigger is working,
- inputs and metadata are recorded.